

## ✓ Day 26: Regularization in TensorFlow

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

# Generating synthetic data for regression
X_train = np.random.rand(200, 1)
y_train = 5 * X_train + np.random.randn(200, 1) * 0.5

# Building a simple model with L1 and L2 regularization
def build_model(regularizer):
    model = tf.keras.Sequential([
        tf.keras.layers.Dense(64, activation='relu', input_shape=(1,),
                                kernel_regularizer=regularizer),
        tf.keras.layers.Dense(1)
    ])
    return model
```

## ✓ L1 and L2 Regularization

L1 Regularization (Lasso): Adds the absolute value of weights to the loss function, encouraging sparsity in the weights. L2 Regularization (Ridge): Adds the squared value of weights to the loss function, helping reduce large weights.

```
# L1 Regularization
l1_model = build_model(tf.keras.regularizers.l1(0.01))
l1_model.compile(optimizer='adam', loss='mean_squared_error')

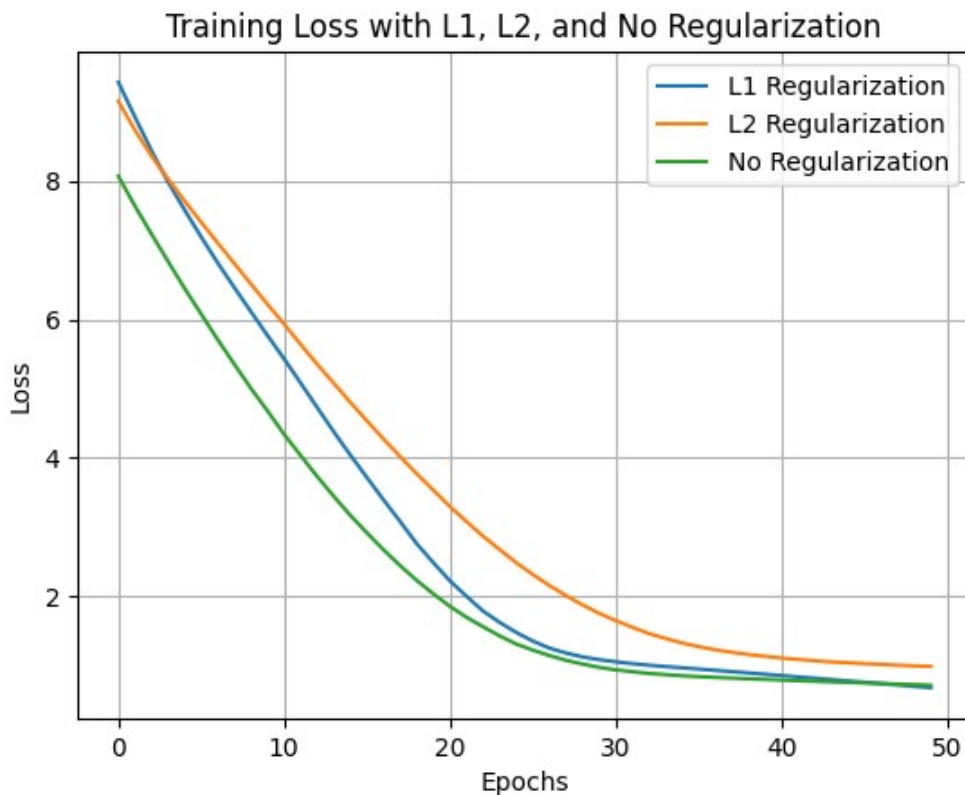
# L2 Regularization
l2_model = build_model(tf.keras.regularizers.l2(0.01))
l2_model.compile(optimizer='adam', loss='mean_squared_error')

# No Regularization (for comparison)
no_reg_model = build_model(None)
no_reg_model.compile(optimizer='adam', loss='mean_squared_error')
```

 /usr/local/lib/python3.10/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not call super().\_\_init\_\_(activity\_regularizer=activity\_regularizer, \*\*kwargs)

```
# Training the models
history_l1 = l1_model.fit(X_train, y_train, epochs=50, verbose=0)
history_l2 = l2_model.fit(X_train, y_train, epochs=50, verbose=0)
history_no_reg = no_reg_model.fit(X_train, y_train, epochs=50, verbose=0)
```

```
# Plotting the loss curves for models with L1, L2, and no regularization
plt.plot(history_l1.history['loss'], label='L1 Regularization')
plt.plot(history_l2.history['loss'], label='L2 Regularization')
plt.plot(history_no_reg.history['loss'], label='No Regularization')
plt.title('Training Loss with L1, L2, and No Regularization')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
plt.show()
```



## ✓ Dropout Regularization

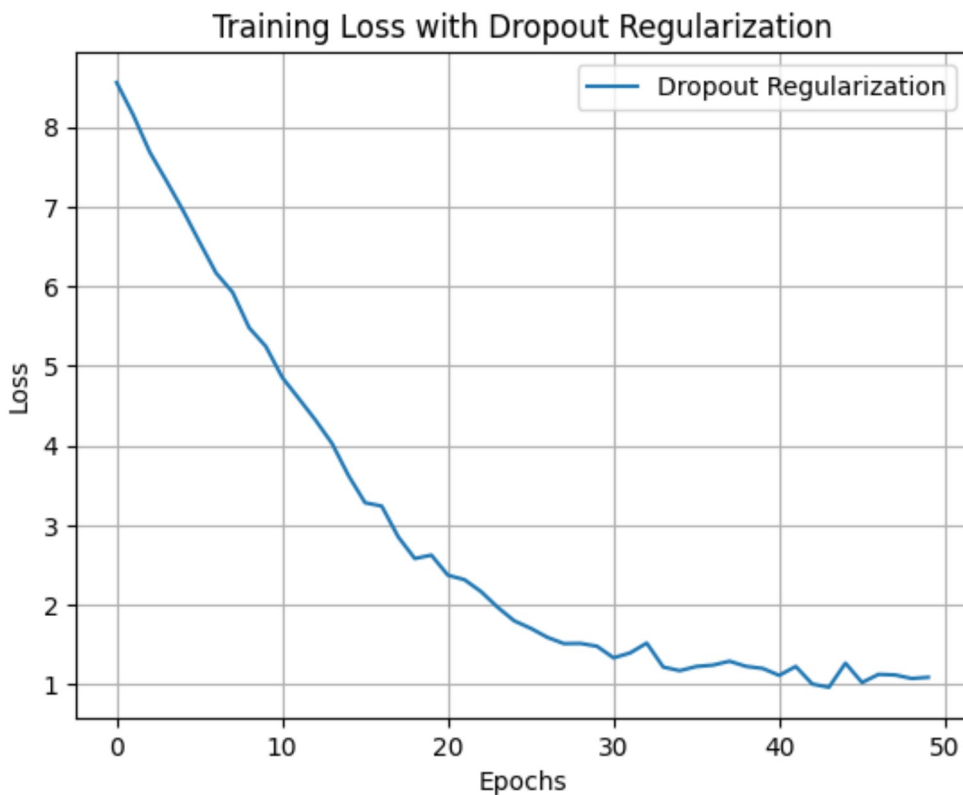
Dropout is another form of regularization, where randomly selected neurons are ignored during training, forcing the network to learn more robust features.

```
# Building a model with Dropout Regularization
dropout_model = tf.keras.Sequential([
    tf.keras.layers.Dense(64, activation='relu', input_shape=(1,)),
    tf.keras.layers.Dropout(0.5), # Dropout with 50% rate
    tf.keras.layers.Dense(1)
])

# Compiling the model
dropout_model.compile(optimizer='adam', loss='mean_squared_error')
```

```
# Training the model with Dropout Regularization
history_dropout = dropout_model.fit(X_train, y_train, epochs=50, verbose=0)
```

```
# Plotting the loss curves for models with Dropout Regularization
plt.plot(history_dropout.history['loss'], label='Dropout Regularization')
plt.title('Training Loss with Dropout Regularization')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
plt.show()
```



- ✓ Regularization techniques like L1, L2, and Dropout help prevent overfitting by reducing model complexity.

Understanding and applying regularization is key to building models that generalize well on unseen data.

Start coding or [generate](#) with AI.

